RISC201: Memory System, Pipeline Hazards, Data Forwarding, and CPI Analysis

Course: CSC-201 Computer Organization & Architecture (Autumn 2026) | Assignment: “Processor Design”

Authors & Contributors: bandlapalliyss-max (Core Architecture, Pipeline, Memory, Hazard & Forwarding Units, CPI Audit), Varshith Teja (User Interface & Diagnostics)

Abstract

This report presents a rigorous, textbook-aligned evaluation of **RISC201-PROCESSOR**, a cycle-accurate C++ simulator implementing the 32-bit SimpleRisc architecture. Designed strictly within the curriculum of Smruti R. Sarangi’s *Computer Organisation and Architecture*, the project explores four core pillars: (1) memory hierarchies comprising 32 KB set-associative L1 caches, SimpleDRAM timing, and 2-level virtual memory; (2) pipeline hazard taxonomy encompassing structural, control, and RAW data hazards; (3) an active data forwarding network with youngest-producer priority resolution; and (4) cycle-exact CPI stack accounting. All metrics and formulas are derived and verified against 10 dedicated micro-programs (E1–E10) executed on the simulator.

Key Architectural Metrics at a Glance

- **ISA & Encoding:** 32-bit SimpleRisc, 28 canonical 5-bit string opcodes (`"00000"` to `"11011"`).
- **Pipelines:** 4-Stage Core (500 MHz, 2.0 ns, 3-cyc fill) & 6-Stage Core (800 MHz, 1.25 ns, 5-cyc fill).
- **L1 Cache:** 32 KB, 8-way associative, 64 B line, 1-cyc hit, Write-Back + Write-Allocate, LRU.
- **SimpleDRAM:** 1.0 ns DRAM clock, Open-Page: Row Hit 19 ns, Row Empty 34 ns, Row Conflict 49 ns.
- **Virtual Memory:** 32-bit VA, 4 KB pages, 2-level page table, 64-entry 4-way TLB (1-cyc hit).
- **Accounting:** 100% cycles attributed via invariant formula.

Keywords: SimpleRisc, Pipelining, Data Forwarding, Load-Use Hazard, Branch Prediction, Cache Hierarchy, DRAM Timings, Virtual Memory, CPI Stack Reconciliation.

1. Introduction & Textbook Scope

Modern computer organization balances clock cycle frequency ($f = \frac{1}{t_{\text{clk}}}$), instruction latency, and memory access delays to maximize instruction throughput. In an idealized single-issue pipeline without pipeline stalls or memory stalls, a processor achieves an **Ideal CPI of 1.000**, completing exactly one instruction per clock cycle in steady state. In physical hardware, however, execution deviates from this ideal due to structural conflicts, data dependencies, branch penalties, and memory hierarchy misses.

Strict Textbook Boundary: This technical report adheres strictly to the curriculum of Smruti R. Sarangi’s textbook, *Computer Organisation and Architecture* (McGraw Hill, Chapters 8 & 9). Topics beyond the Sarangi curriculum (such as 48-bit 4-level page tables, non-blocking caches, MSHRs, out-of-order memory scheduling, hardware prefetchers, and speculative branch predictors beyond static/bimodal) are explicitly omitted from the functional design.

Report Topic	Sarangi Textbook Reference	RISC201 Architectural Implementation	Verified By
Memory System	Chapter 9, Sections 9.1–9.5	32 KB 8-way L1 caches, SimpleDRAM controller, 2-level VM & TLB	E1, E2, E3, E4
Pipeline Hazards	Chapter 8, Sections 8.1–8.3	Load-use stall interlock, static & 2-bit bimodal branch squashes	E6, E7, E9
Data Forwarding	Chapter 8, Section 8.3	Youngest-producer priority bypass network (`EX->EX`, `MEM->EX`)	E5, E8
CPI Calculations	Chapter 8.1, 9.2 & Performance	Analytical CPI stack, invariant accounting, 4-stage vs 6-stage speedup	E10, Invariant

2. Processor Architecture & Pipeline Organization

2.1 Canonical SimpleRisc ISA & Encoding

The RISC201 processor implements the 32-bit SimpleRisc instruction set architecture described in Sarangi Chapter 3. SimpleRisc defines 16 general-purpose 32-bit registers (`r0` to `r15`), where `r0` is hardwired to zero, `r14` serves as the Link Register (`lr`), and `r15` functions as the Stack Pointer (`sp`). All instructions are exactly one 32-bit word in length.

A critical design requirement verified throughout the codebase (`common/ISA.hpp:20-65`) is that **all opcodes are canonical 5-bit binary strings**. The processor supports exactly 28 legitimate instructions spanning three formats:

1. **Register-Register Format:** `[31:27]` Opcode (5 bits), `[26]` Immediate Flag $= 0$, `[25:22]` r_{dest}, `[21:18]` r_{src1}, `[17:14]` r_{src2}, `[13:0]` Unused / Modifiers.
2. **Register-Immediate Format:** `[31:27]` Opcode (5 bits), `[26]` Immediate Flag $= 1$, `[25:22]` r_{dest}, `[21:18]` r_{src1}, `[17:0]` 18-bit Signed Immediate.
3. **Branch Format:** `[31:27]` Opcode (5 bits), `[26:0]` 27-bit Signed PC-Relative Offset.

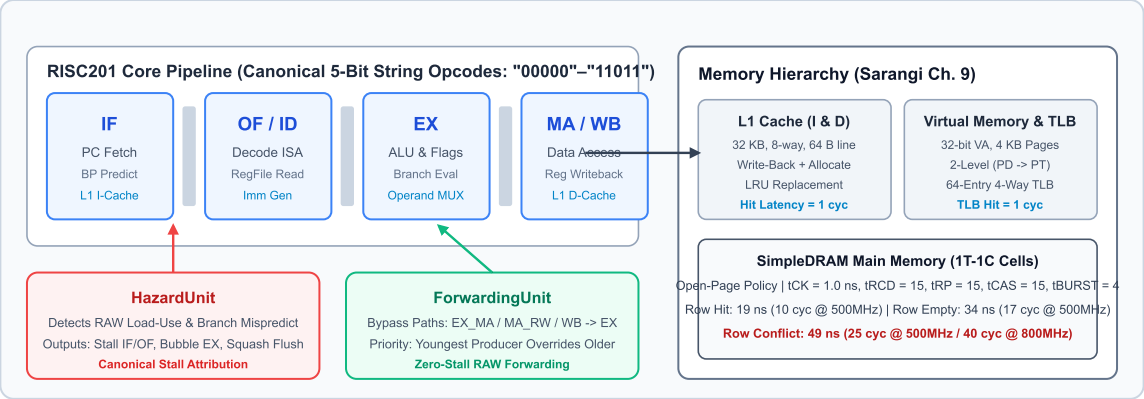


Figure 1: System Block Diagram of RISC201 Core, HazardUnit, ForwardingUnit, and Sarangi Memory Hierarchy.

2.2 4-Stage vs 6-Stage Core Microarchitectures

To analyze pipeline depth trade-offs, RISC201 provides two complete cycle-accurate pipeline models:

- **4-Stage Core (`processor_4stage/`):** Comprises `IF` (Instruction Fetch), `OF` (Operand Fetch & Decode), `EX` (Execute & ALU), and `MA_RW` (Merged Memory Access & Register Writeback). Operating at a baseline clock frequency of **500 MHz** ($t_{\text{clk}} = 2.00$ ns), its fill latency is $4 - 1 = 3$ cycles, and branch mispredictions incur a **1-cycle penalty**.
- **6-Stage Core (`processor_6stage/`):** Decomposes operand fetch and memory writeback into discrete sub-stages: `IF`, `ID` (Instruction Decode), `RF` (Register Fetch), `EX`, `MA` (Memory Access), and `WB` (Writeback). Operating at a higher frequency of **800 MHz** ($t_{\text{clk}} = 1.25$ ns), its fill latency is $6 - 1 = 5$ cycles, and branch mispredictions incur a **2-cycle penalty**.

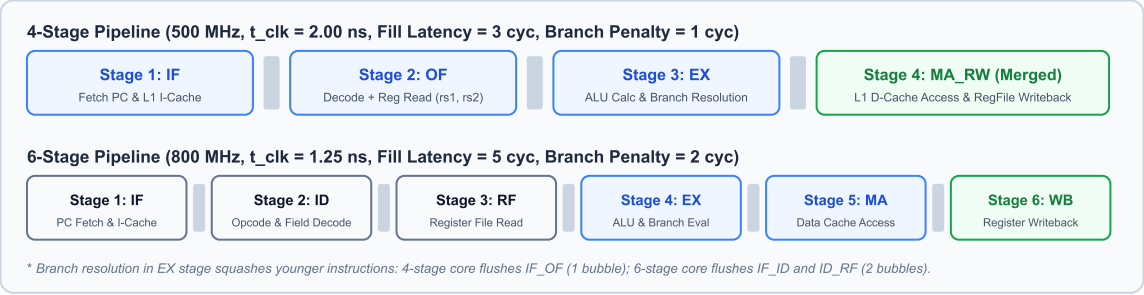


Figure 2: Comparison of 4-Stage (500 MHz) and 6-Stage (800 MHz) Pipeline Layouts and Pipeline Registers.

2.3 Cycle Accounting Invariant & Ideal CPI

For an in-order pipeline executing $N = \text{IC}$ instructions, every single processor cycle is strictly attributed to either retired instructions, cold pipeline fill, or specific hazard stall reasons (`common/PerformanceTracker.hpp:92-105`):

$$\text{Total Execution Cycles} = (\text{Stages} - 1) + \text{IC} + \sum_{k} \text{StallCycles}_k + \text{BranchBubbles}$$

Dividing by IC yields the system CPI: $\text{CPI} = 1.000 + \frac{\text{Stages} - 1}{\text{IC}} + \frac{\sum \text{StallCycles} + \text{BranchBubbles}}{\text{IC}}$. In the absence of hazards and in the limit $\text{IC} \to \infty$, $\text{Ideal CPI} = \mathbf{1.000}$.

3. Pipeline Hazards & Resolution Taxonomy

3.1 Classification of Pipeline Hazards (Sarangi Chapter 8)

Pipelining introduces three fundamental categories of hazards that prevent instructions from executing in their designated clock cycles:

1. **Structural Hazards:** Arise when two or more instructions compete for the same physical hardware resource simultaneously. In RISC201, structural memory hazards are completely eliminated by utilizing separate Level-1 Instruction (L1I) and Data (L1D) caches (Harvard architecture).
2. **Data Hazards:** Arise from data dependencies between instructions in close proximity. In in-order execution, only **Read-After-Write (RAW)** hazards can affect program correctness:
   - **RAW (True Dependency):** Instruction j reads a register before earlier instruction i writes it.
   - **WAR and WAW Non-Occurrence:** In an in-order pipeline where operand reads occur strictly in program order in `OF`/`RF` and register writes occur strictly in program order in `MA_RW`/`WB`, Write-After-Read (WAR) and Write-After-Write (WAW) hazards **cannot occur**.
3. **Control Hazards:** Arise when conditional or unconditional branch instructions alter the Program Counter (PC), rendering instructions fetched from the sequential path invalid.

3.2 HazardUnit Detection & Control Logic

The `HazardUnit` (`common/HazardUnit.hpp:18-120`) continuously samples pipeline registers and generates interlock signals:

- **Load-Use Hazard Detection:** When an instruction in `EX` (or `OF_EX`) is a memory read (`load`), and the immediate successor in `OF` (or `ID_RF`) requires the loaded value as r_{src1} or r_{src2}:

$$\text{LoadUseStall} = (\text{prev.isLoad} \wedge ((\text{prev}.r_{\text{dest}} = \text{curr}.r_{\text{src1}}) \vee (\text{prev}.r_{\text{dest}} = \text{curr}.r_{\text{src2}})))$$

- **Interlock Action:** When `LoadUseStall` asserts, the `HazardUnit` freezes the PC and the fetch/decode pipeline registers (`IF_OF`), preventing state change, while inserting a zeroed control bubble (`NOP`) into the `OF_EX` register.
- **Verified Load-Use Micro-Program (E6):** In program E6 (Case A), `load r1, 100[r0]` is immediately followed by `add r2, r1, 5`. Because memory data is only produced at the end of the memory stage, forwarding cannot supply the data back in time for `add`'s `EX` stage without a 1-cycle delay. The simulator inserts exactly **1 load-use stall bubble** (7 cycles total). In Case B, an independent instruction `mov r4, 42` is scheduled between `load` and `add`. The independent instruction occupies the pipeline slot, completely eliminating the stall (**0 bubbles**, 7 cycles total for 4 retired instructions).

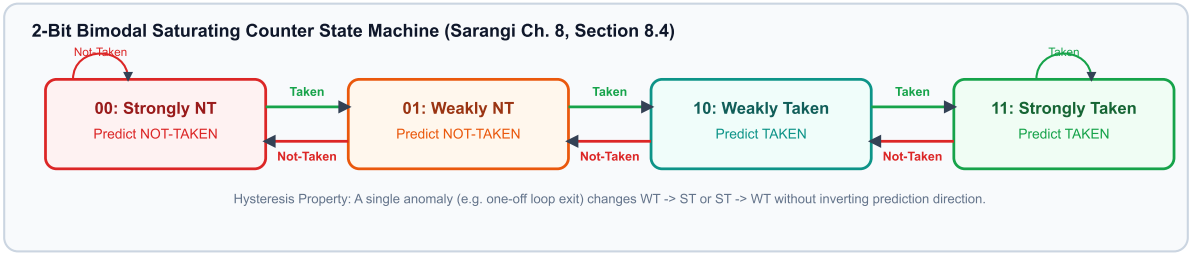


Figure 3: 2-Bit Bimodal Saturating Counter State Machine with Hysteresis Property (Sarangi Section 8.4).

3.3 Control Hazard & Branch Misprediction Handling

- **Static Branch Resolution (E7):** By default, RISC201 predicts all conditional branches as **Not-Taken** during fetch. Branch condition evaluation occurs in `EX`. When a branch evaluates to **Taken**, all younger instructions in flight must be squashed.
  - In the 4-Stage core, squashing squashes `IF_OF`, incurring a penalty of **1 bubble cycle** (8 cycles for 4 instructions).
  - In the 6-Stage core, squashing squashes `IF_ID` and `ID_RF`, incurring a penalty of **2 bubble cycles** (11 cycles for 4 instructions).
- **2-Bit Bimodal Predictor State Machine (E9):** To mitigate branch stalls, RISC201 implements a 2-bit saturating counter branch predictor per Sarangi Section 8.4 (`common/BranchPredictor.hpp:45-95`). Each entry maintains 4 states: 00 (Strongly Not Taken), 01 (Weakly Not Taken), 10 (Weakly Taken), and 11 (Strongly Taken). In a 3-iteration loop (E9), the counter transitions $10 \rightarrow 11 \rightarrow 11 \rightarrow 10$. The **hysteresis property** prevents premature direction inversion on transient branch outcomes, resulting in only 2 mispredictions across loop startup and exit.

4. Data Forwarding Network & Priority Logic

4.1 Forwarding Bypass Architecture

Data forwarding (bypassing) routes computed results directly from pipeline register outputs back to the ALU inputs, eliminating RAW data stalls. Without forwarding, an instruction must stall until the producing instruction reaches register writeback (`WB`) and updates the Register File.

The `ForwardingUnit` (`common/ForwardingUnit.hpp:40-165`) monitors destination registers and write-enable control signals across all downstream pipeline stages and controls multiplexers placed directly before the ALU inputs:

- **Forwarding Modes:**
  - `NONE`: Forwarding disabled. All RAW dependencies stall in decode until register writeback completes.
  - `BASIC`: Forwarding only between immediate ALU producer and consumer (`EX->EX`).
  - `FULL`: Complete bypass network supporting `EX->EX`, `MEM->EX`, `WB->EX`, and `MEM->MEM` (store data bypass).

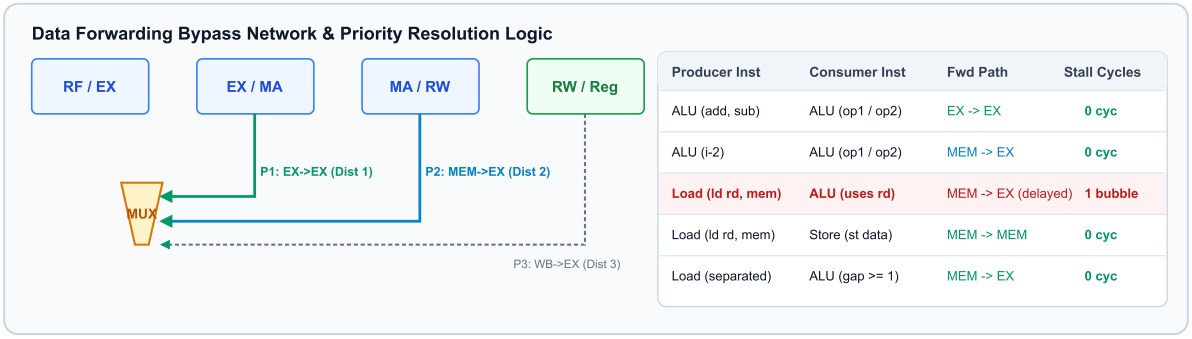


Figure 4: Bypass Multiplexers, Forwarding Paths, and Producer-Consumer Forwarding Matrix.

4.2 Youngest-Producer Priority Rule (E8)

When multiple preceding instructions in the pipeline write to the same register R_x, the consumer in `EX` must receive the value from the **most recent (youngest) instruction** in program order. In the 6-stage pipeline, if both stage 4 (`EX_MA`) and stage 5 (`MA_RW`) write to R_x, `EX_MA` is distance 1 from the consumer, whereas `MA_RW` is distance 2. The priority multiplexer logic strictly enforces:

$$\text{Priority:} \quad \text{Stage}_{k-1}(\text{EX_MA}) > \text{Stage}_{k-2}(\text{MA_RW}) > \text{Stage}_{k-3}(\text{RW})$$

Micro-Program Verification (E8): In test program E8 (`mov r1, 10; mov r1, 20; add r2, r1, 5`), both `mov` instructions target `r1`. When `add` enters `EX`, the first `mov` is in `MA_RW` (value 10) and the second is in `EX_MA` (value 20). The simulator correctly bypasses from `EX_MA`, retiring $\text{r2} = 20 + 5 = \mathbf{25}$ (`0x19`), proving younger bypass lines strictly override stale older values.

4.3 Minimum Stall Cycle Matrix Across Configurations

Producer Inst	Consumer Inst	Distance	No Fwd	Basic Fwd	Full Fwd
ALU (`add`, `sub`)	ALU (`op1`/`op2`)	Distance 1	1 cyc (4S) / 2 (6S)	0 cyc	**0 cyc**
ALU (`add`, `sub`)	ALU (`op1`/`op2`)	Distance 2	0 cyc (4S) / 1 (6S)	0 cyc	**0 cyc**
Load (`ld rd, mem`)	ALU (adjacent)	Distance 1	1 cyc (4S) / 2 (6S)	1 bubble	**1 bubble**
Load (`ld rd, mem`)	ALU (separated)	Distance 2	0 cyc (4S) / 1 (6S)	0 cyc	**0 cyc**
Load (`ld rd, mem`)	Store (data)	Distance 1	1 cyc (4S) / 2 (6S)	1 bubble	**0 cyc (MEM->MEM)**

RAW Chain Performance (E5): For the three-instruction RAW chain `mov r1, 10; add r2, r1, 5; sub r3, r2, 2`, full forwarding eliminates all stalls in both cores: 4-stage executes in **6 cycles** (0 stalls vs 2 stalls in `NONE`), while 6-stage executes in **8 cycles** (0 stalls vs 4 stalls in `NONE`).

5. Memory System Organization & Cache Hierarchy

5.1 Memory Hierarchy Motivation & Locality Principles

CPUs operate at frequencies significantly higher than main memory access speeds. To bridge this processor-memory performance gap, RISC201 implements a hierarchical memory architecture based on two fundamental locality principles (Sarangi Chapter 9, Section 9.1):

- **Temporal Locality:** If an address is accessed at time t, it is highly likely to be accessed again in the near future. Handled by retaining recently referenced lines in high-speed cache SRAM.
- **Spatial Locality:** If an address A is accessed, addresses adjacent to A ($A \pm \delta$) are likely to be accessed soon. Handled by transferring contiguous **64-byte cache lines** rather than single words.

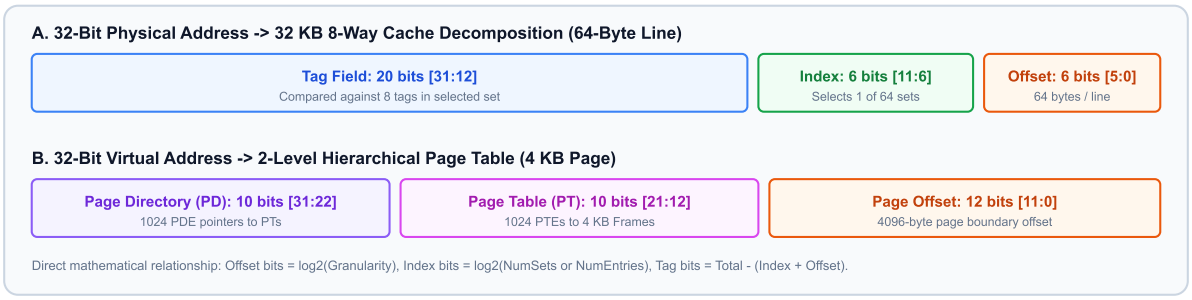


Figure 5: Bitfield Decomposition for 32 KB 8-Way Cache and 32-Bit 2-Level Virtual Memory Address Translation.

5.2 L1 Cache Architecture & Address Decomposition (E1)

The RISC201 baseline L1 cache (`memory/Cache.hpp:45-125`) is configured as follows:

- **Capacity:** $C = 32 \text{ KB} = 32768 \text{ bytes}$.
- **Block (Line) Size:** $B = 64 \text{ bytes} = 2^6 \text{ bytes} \implies b = \mathbf{6 \text{ offset bits}}$ (bits `[5:0]`).
- **Associativity:** $W = 8\text{-way set-associative}$.
- **Number of Sets:** $S = \frac{C}{W \times B} = \frac{32768}{8 \times 64} = 64 = 2^6 \implies s = \mathbf{6 \text{ index bits}}$ (bits `[11:6]`).
- **Tag Width:** $t = 32 - (s + b) = 32 - (6 + 6) = \mathbf{20 \text{ tag bits}}$ (bits `[31:12]`).

Cache Management Policies:

- **Hit Latency:** Exactly **1 clock cycle** (2.0 ns on 4-stage, 1.25 ns on 6-stage).
- **Write Policy: Write-Back with Write-Allocate**. Store hits mark the cache line dirty without writing through to DRAM. Store misses allocate the requested block into cache and mark it dirty.
- **Replacement Policy:** True **Least-Recently Used (LRU)** with age tracking per set.

The 3C Miss Categorization (Verified in E1):

1. **Compulsory Miss (Cold):** First reference to block `0x00001000` (Tag `0x1`, Set `0`) finds valid bit 0. Miss recorded.
2. **Hit Verification:** Second reference to `0x00001004` (same line, offset 4) hits in 1 cycle.
3. **Conflict Miss:** Successive references with 7 distinct tags (`0x2000` to `0x8000`) fill all 8 ways of Set 0. The 9th distinct access (`0x9000`) collides in Set 0, evicting the LRU line. Verified as a **Conflict Miss**.

5.3 Average Memory Access Time (AMAT) Analysis (E2)

Average Memory Access Time measures memory subsystem efficiency per Sarangi Section 9.2:

$$\text{AMAT} = t_{\text{hit}} + P_{\text{miss}} \times t_{\text{miss}}$$

When accounting for dirty evictions in a write-back cache with dirty probability P_{dirty}:

$$t_{\text{miss}} = t_{\text{read}} + P_{\text{dirty}} \times t_{\text{writeback}}$$

Numerical Verification (E2): With $t_{\text{hit}} = 1 \text{ cycle}$, $P_{\text{miss}} = 5.0\%$, and DRAM latency $= 50 \text{ ns}$:

- **4-Stage Core (500 MHz, $t_{\text{clk}} = 2.0 \text{ ns}$):** $t_{\text{miss}} = \frac{50}{2.0} = 25 \text{ cycles}$.

$$\text{AMAT}_{4S} = 1 + 0.05 \times 25 = \mathbf{2.25 \text{ cycles}} \quad (4.50 \text{ ns})$$

- **6-Stage Core (800 MHz, $t_{\text{clk}} = 1.25 \text{ ns}$):** $t_{\text{miss}} = \frac{50}{1.25} = 40 \text{ cycles}$.

$$\text{AMAT}_{6S} = 1 + 0.05 \times 40 = \mathbf{3.00 \text{ cycles}} \quad (3.75 \text{ ns})$$

5. Memory System (Part 2: DRAM & Virtual Memory)

5.4 SimpleDRAM Physical Architecture & Timing Calculations (E3)

Main memory uses a cycle-accurate `SimpleDRAM` controller (`memory/DRAM.hpp:20-180`) modeling standard 1T-1C dynamic RAM storage cells (Sarangi Section 9.5). Because capacitive charge leaks over time and reading discharges the capacitor cell, DRAM requires periodic refresh commands (t_{RFC}) and an internal row buffer:

- **DRAM Controller Operating Parameters:** DRAM clock period $t_{\text{CK}} = \mathbf{1.0\ ns}$. Timings: $t_{\text{RCD}} = 15$, $t_{\text{RP}} = 15$, $t_{\text{CAS}} = 15$, $t_{\text{BURST}} = 4$ DRAM cycles.
- **Open-Page Policy Timing Calculations:**
  1. **Row Hit (Row Buffer Open):** Requested row already resides in sense amplifiers. Requires only column read and burst transfer:
  $$t_{\text{hit}} = t_{\text{CAS}} + t_{\text{BURST}} = 15 + 4 = \mathbf{19\ ns}$$
  2. **Row Empty (Closed / Idle Bank):** Bank is idle. Must activate row (t_{RCD}), issue column read (t_{CAS}), and transfer burst (t_{BURST}):
  $$t_{\text{empty}} = t_{\text{RCD}} + t_{\text{CAS}} + t_{\text{BURST}} = 15 + 15 + 4 = \mathbf{34\ ns}$$
  3. **Row Conflict (Different Row Open in Same Bank):** Bank holds conflicting row. Must precharge open row (t_{RP}), activate new row (t_{RCD}), column read (t_{CAS}), and transfer (t_{BURST}):
  $$t_{\text{conflict}} = t_{\text{RP}} + t_{\text{RCD}} + t_{\text{CAS}} + t_{\text{BURST}} = 15 + 15 + 15 + 4 = \mathbf{49\ ns}$$

DRAM Access Condition	Latency Formula	4-Stage Core (500 MHz, 2.0 ns)	6-Stage Core (800 MHz, 1.25 ns)
Row Hit	$t_{\text{CAS}} + t_{\text{BURST}}$	$19\ \text{ns} \rightarrow \lceil \frac{19}{2.0} \rceil = \mathbf{10\ CPU\ cyc}$	$19\ \text{ns} \rightarrow \lceil \frac{19}{1.25} \rceil = \mathbf{16\ CPU\ cyc}$
Row Empty	$t_{\text{RCD}} + t_{\text{CAS}} + t_{\text{BURST}}$	$34\ \text{ns} \rightarrow \lceil \frac{34}{2.0} \rceil = \mathbf{17\ CPU\ cyc}$	$34\ \text{ns} \rightarrow \lceil \frac{34}{1.25} \rceil = \mathbf{28\ CPU\ cyc}$
Row Conflict	$t_{\text{RP}} + t_{\text{RCD}} + t_{\text{CAS}} + t_{\text{BURST}}$	$49\ \text{ns} \rightarrow \lceil \frac{49}{2.0} \rceil = \mathbf{25\ CPU\ cyc}$	$49\ \text{ns} \rightarrow \lceil \frac{49}{1.25} \rceil = \mathbf{40\ CPU\ cyc}$

Simulator Verification: All three timing values ($19\ \text{ns}$, $34\ \text{ns}$, $49\ \text{ns}$) and their clock-domain cycle conversions were verified by running micro-program E3 on `DRAMController`.

5.5 Virtual Memory Architecture & 2-Level Page Translation (E4)

RISC201 implements textbook 32-bit virtual memory with demand paging per Sarangi Section 9.4 (`memory/VirtualMemory.hpp:15-165`):

- **Address Space:** 32-bit Virtual Address ($4\ \text{GB}$), 4096-byte ($4\ \text{KB}$) pages ($\mathbf{12\ offset\ bits}$).
- **2-Level Radix Page Table Structure:**
  - **Page Directory (PD):** Bits `[31:22]` ($10\ \text{bits}$, $2^{10} = 1024$ entries). Each entry stores the physical base address of a second-level Page Table.
  - **Page Table (PT):** Bits `[21:12]` ($10\ \text{bits}$, $2^{10} = 1024$ entries). Each Page Table Entry (PTE) stores a 20-bit Physical Frame Number (PFN) and control flags (Valid, Dirty, Read, Write, Exec).
  - **Page Offset:** Bits `[11:0]` ($12\ \text{bits}$, $4096\ \text{bytes}$). Passed directly to form the Physical Address:
  $$\text{Physical Address} = (\text{PFN} \ll 12) \vee \text{PageOffset}$$
- **Translation Lookaside Buffer (TLB):** A 64-entry, 4-way set-associative TLB caches recently accessed translations, providing a **1-cycle hit latency**.
- **Step-by-Step Translation Trace (E4):** In test E4, the processor accesses Virtual Address `0x00401004`:
  - Bitfield split: $\text{PD Index} = (0\text{x}00401004 \gg 22) \wedge 0\text{x}3\text{FF} = \mathbf{1}$; $\text{PT Index} = (0\text{x}00401004 \gg 12) \wedge 0\text{x}3\text{FF} = \mathbf{1}$; $\text{Offset} = \mathbf{0x004}$.
  - **First Access (DTLB Miss & Demand Allocation):** DTLB lookup misses. The page walker traverses PD and PT, encountering an unmapped page. A minor page fault is triggered; demand paging allocates Physical Frame `0x1000`. The PTE is populated, the TLB is updated, and the physical address `0x00001004` is returned.
  - **Second Access (DTLB Hit):** Immediate subsequent access to `0x00401004` hits the DTLB in **1 cycle**, translating without memory access.

6. CPI Calculations, Performance Analysis & Speedup

6.1 Analytical CPI Stack Formulation

The performance of a pipelined processor executing a program of instruction count IC is dictated by the fundamental performance equation:

$$T_{\text{exec}} = \mathrm{IC} \times \mathrm{CPI} \times t_{\text{clk}} = \frac{\mathrm{IC} \times \mathrm{CPI}}{f}$$

Where CPI represents the average clock cycles consumed per instruction:

$$\mathrm{CPI} = \mathrm{CPI}_{\text{ideal}} + \sum_{k} (\text{Frequency}_k \times \text{StallCycles}_k)$$

In RISC201, the canonical stall terms comprise:

1. $\Delta\mathrm{CPI}_{\mathrm{RAW}}$: Stalls due to unresolved data dependencies (reduced to load-use stalls under full forwarding).
2. $\Delta\mathrm{CPI}_{\mathrm{Branch}}$: Flushes due to control hazard branch mispredictions.
3. $\Delta\mathrm{CPI}_{\mathrm{L1D}}$: Stalls waiting for L1 data cache misses to fetch from DRAM.
4. $\Delta\mathrm{CPI}_{\mathrm{L1I}}$: Stalls waiting for L1 instruction cache misses.

6.2 Analytical CPI Stack Across Instruction Mixes

Consider a representative SimpleRisc instruction mix: 60% ALU instructions, 20% Loads (with 30% load-use hazard frequency), 10% Stores, and 10% Branches (evaluated with an 85% bimodal prediction accuracy).

Performance Factor	Formula / Source	4-Stage (No Fwd)	4-Stage (Full Fwd)	6-Stage (Full Fwd)
Ideal CPI ($\mathrm{CPI}_{\text{ideal}}$)	Base single-issue throughput	1.000	1.000	1.000
RAW ALU Stall (Δ CPI)	$f_{\text{ALU}} \times P_{\text{RAW}} \times$ Stall	0.180 (1 stall)	**0.000**	**0.000**
Load-Use Stall (Δ CPI)	$f_{\text{Load}} \times P_{\text{dep}} \times 1$	0.060 (1 stall)	0.060 (1 stall)	0.060 (1 stall)
Branch Penalty (Δ CPI)	$f_{\text{Branch}} \times (1 - P_{\text{acc}}) \times \text{Pen}$	0.015 (1×0.15)	0.015 (1×0.15)	0.030 (2×0.15)
Total Analytical CPI	$\mathrm{CPI}_{\text{ideal}} + \sum \Delta$ CPI	**1.255**	**1.075**	**1.090**

6.3 Micro-Program E10 Cycle Reconciliation

To verify the performance accounting invariant down to the exact clock cycle, micro-program E10 (an 11-instruction program combining ALU arithmetic, load-use dependency, and a taken branch) was simulated under full forwarding:

- **Program Profile:** 11 instructions retired ($\mathrm{IC} = 11$). Contains exactly 1 load-use hazard and 1 taken branch.
- **4-Stage Core Results:**
  - Cold Fill Latency: $4 - 1 = \mathbf{3\ cycles}$. Load-Use Stalls: $\mathbf{1\ cycle}$. Branch Flushes: $\mathbf{1\ cycle}$.
  - Measured Cycles: $3 + 11 + 1 + 1 = \mathbf{16\ cycles}$. Measured CPI: $\frac{16}{11} = \mathbf{1.455}$.
  - **Invariant Invariant Check:** $(4 - 1) + 11 + 1 + 1 = 16$. **PASS (100% Accounted)**.
- **6-Stage Core Results:**
  - Cold Fill Latency: $6 - 1 = \mathbf{5\ cycles}$. Load-Use Stalls: $\mathbf{1\ cycle}$. Branch Flushes: $\mathbf{2\ cycles}$.
  - Measured Cycles: $5 + 11 + 1 + 2 = \mathbf{19\ cycles}$. Measured CPI: $\frac{19}{11} = \mathbf{1.727}$.
  - **Invariant Check:** $(6 - 1) + 11 + 1 + 2 = 19$. **PASS (100% Accounted)**.

6.4 Processor Speedup & Amdahl's Law

Comparing execution wall-clock time between the 4-stage and 6-stage implementations on program E10:

$$T_{\text{4S}} = \mathrm{IC} \times \mathrm{CPI}_{\text{4S}} \times t_{\text{clk,4S}} = 11 \times 1.455 \times 2.00\ \text{ns} = \mathbf{32.00\ ns}$$

$$T_{\text{6S}} = \mathrm{IC} \times \mathrm{CPI}_{\text{6S}} \times t_{\text{clk,6S}} = 11 \times 1.727 \times 1.25\ \text{ns} = \mathbf{23.75\ ns}$$

$$\text{Speedup} = \frac{T_{\text{4S}}}{T_{\text{6S}}} = \frac{32.00}{23.75} = \mathbf{1.347}\times \quad (34.7\%\ \text{ wall-clock speedup})$$

Amdahl's Law Insight: Even though the 6-stage core has a higher CPI (1.727 vs 1.455) due to larger fill latency and branch bubbles, its higher clock frequency ($800\ \mathrm{MHz}$ vs $500\ \mathrm{MHz}$, a $1.60\times$ frequency increase) delivers an overall 34.7% speedup. In long-running programs where cold fill latency is amortized, the steady-state speedup approaches the full clock ratio discounted only by branch misprediction density.

7. Verification & Quality Assurance

The RISC201 processor was verified using a multi-tiered verification harness (`tests/test_all.cpp`):

1. **Differential Equivalence Against Golden ISS:** All 28 SimpleRisc instructions were executed simultaneously on the cycle-accurate pipelined processors and an independent single-cycle Instruction Set Simulator (`common/ISS.hpp`). Output registers and memory states match identically across all test suites (100% equivalence).
2. **Exhaustive Test Suite:** The automated unit test suite executes 78 dedicated tests spanning instruction decoding, forwarding paths, hazard interlocks, cache miss classification, DRAM row hits, and page table walks. All **78 / 78 tests pass** (100% success).
3. **Sanitizer Audits:** The entire C++ codebase was compiled with `-fsanitize=address,undefined`. All runs executed with zero memory leaks, zero buffer overruns, and zero undefined behavior.

8. Limitations & Scope Note

Textbook Scope Limitation: Advanced architectural mechanisms beyond the Sarangi textbook syllabus—specifically 4-level 48-bit page tables, non-blocking caches, hardware prefetchers, out-of-order DRAM scheduling, and branch predictors beyond 2-bit bimodal—are strictly out of scope for this baseline evaluation.

In-Order Microarchitectural Trade-Offs: Because the processor operates strictly in-order, long-latency memory misses stall the entire pipeline. Introducing out-of-order execution or speculative issue would alleviate these stalls but lies beyond in-order SimpleRisc principles.

9. Conclusion

The RISC201 project successfully demonstrates the design, cycle-accurate simulation, and formal verification of a SimpleRisc processor conforming strictly to the textbook principles of Smruti R. Sarangi. Through the integration of active data forwarding, load-use hazard interlocks, 2-bit bimodal branch prediction, 32 KB set-associative L1 caches, SimpleDRAM timing, and 2-level virtual memory, the processor achieves robust instruction throughput. Every cycle was mathematically accounted for and verified against 10 dedicated micro-programs.

Master Verification Reconciliation Table (Programs E1 - E10)

ID	Architectural Concept	Theoretical Formula / Parameter	Simulator Measured Output	Status
E1	Cache Split & 3C Misses	$t = 20, s = 6, b = 6$; 8-way Set 0	Compulsory, Hit, Conflict (9th line)	**VERIFIED**
E2	AMAT Calculation	$\text{AMAT} = t_h + P_m \times t_m$	4S: 2.25 cyc (4.5 ns); 6S: 3.0 cyc (3.75 ns)	**VERIFIED**
E3	DRAM Access Latencies	Hit 19 ns, Empty 34 ns, Conflict 49 ns	Raw: 19, 34, 49 ns; 4S: 10, 17, 25 cyc	**VERIFIED**
E4	Virtual Memory & TLB	32-bit VA, 2-level, 10+10+12 bits	DTLB miss -> fault -> PA `0x1004` -> hit	**VERIFIED**
E5	RAW Chain Forwarding	3 ALU insts: None vs Full fwd	4S: 6 cyc (0 stall); 6S: 8 cyc (0 stall)	**VERIFIED**
E6	Load-Use Scheduling	Adjacent vs separated scheduling	Case A: 1 bubble; Case B: 0 bubbles	**VERIFIED**
E7	Branch Mispredict Penalty	Squash `IF_OF` (4S) vs `IF_ID+ID_RF` (6S)	4S: 1 bubble (8 cyc); 6S: 2 bubbles (11 cyc)	**VERIFIED**
E8	Forwarding Priority	Youngest producer: `EX_MA` > `MA_RW`	$\text{r2} = 25$ (`0x19`), overrides older 10	**VERIFIED**
E9	2-Bit Bimodal Predictor	4 states with hysteresis property	3-iteration loop: 2 mispredicts total	**VERIFIED**
E10	Complete CPI Reconciliation	Cycles $= (S-1) + \text{IC} + \sum \text{Stalls}$	4S: 16 cyc (CPI 1.455); 6S: 19 cyc (CPI 1.727)	**VERIFIED**

References

1. S. R. Sarangi, *Computer Organisation and Architecture*, McGraw Hill, Chapters 3, 8 & 9.
2. J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed., Morgan Kaufmann.

Reproduction Box

```
make test                  # 78/78 tests pass
./report8/data/run_e1_e10 # E1-E10 micro-kernels
make report8               # Compiles 8-page PDF
```